

Module 7: Functions II

Introduction to C++ Programming — A Beginner's Course

Module Overview

Four distinct upgrades to functions land in one module: pass-by-reference, default parameters, overloading, and recursion. Of these, recursion is conceptually the hardest thing in the course so far and deserves the most class time — budget accordingly rather than splitting time evenly across all four topics.

Suggested Duration: 90 minutes — recommend two sessions if possible: (reference/defaults/overloading) then (recursion, given its own full session)

Learning Objectives

By the end of this module, students will be able to:

- Use reference parameters (&) to let a function modify the caller's variable.
- Explain why reference parameters are the standard way to return more than one value.
- Use default parameter values correctly (including the last-parameter-only rule).
- Write and correctly distinguish overloaded functions.
- Write a recursive function with a correct, reachable base case.
- Trace a simple recursive call by hand, step by step.

Prerequisites

- Module 6: solid grasp of pass-by-value, since references are explicitly the deliberate exception to it.

Materials & Instructor Prep

- 07-functions-ii/examples/03_recursion.cpp preloaded.
- Whiteboard space reserved specifically for hand-tracing factorial(4) — this needs real board space, not just a slide.

Key Vocabulary

Term	Definition
Reference parameter (&)	A parameter that is another name for the caller's own variable, not a copy — changes inside the function affect the original.
Default parameter	A parameter with a value used automatically when the caller omits that argument.
Function overloading	Multiple functions sharing a name but differing in their parameter lists; the compiler picks the right one based on the call.
Recursion	A function that calls itself to solve a smaller version of the same problem.
Base case	The condition in a recursive function simple enough to answer directly, without further recursion — required to stop the recursion.
Stack overflow	A crash caused by recursion (or any call chain) going too deep, typically from a missing or unreachable base case.

Lesson Outline & Instructor Talking Points

1. Pass by reference — the deliberate exception (20 min)

- Open by explicitly recalling Module 6: 'Last time, a function couldn't change the caller's variable — that was a safety feature. Today we learn the on-purpose way around it.'
- Live-code `doublelt(int &x)` and show it DOES change the caller's variable this time — contrast directly against the Module 6 pass-by-value example on a split screen or two side-by-side terminals if possible.
- Introduce the `minMax(vector, int&, int&)` pattern as the practical motivation: a function can only return ONE value with `return`, but can update several via reference parameters.

Instructor note: *The direct side-by-side contrast with Module 6's pass-by-value demo is the highest-leverage moment here — don't skip re-running that old example for comparison.*

2. Default parameters (10 min)

- Show `greet(name, greeting = "Hello")`. Call it both ways (with and without the second argument).
- State the ordering rule explicitly and show what happens if you try to violate it (defaulted parameter before a non-defaulted one) — let the compiler error reinforce the rule.

3. Function overloading (15 min)

- Show two `add()` overloads (int and double versions). Call both and ask students to predict, before running, which version handles each call.

- Connect back to something they already trust: 'This is exactly how `std::cout <<` works no matter what type you give it — it's overloaded behind the scenes for every type.'

4. Recursion — build intuition slowly (30–35 min)

- Do NOT start with code. Start with a plain-language recursive definition students already intuitively understand, e.g. 'to find the height of a line of people, ask the last person their height, then add everyone in front of them the same way' — or trace factorial verbally first: $4! = 4 \times 3!$, $3! = 3 \times 2!$, and so on.
- THEN show the code:

```
int factorial(int n) { if (n <= 1) return 1; return n * factorial(n-1); }
```
- Hand-trace `factorial(4)` on the board, expanding every nested call fully before collapsing back up with the answer — this expand-then-collapse visual is the single most important artifact of the lesson.
- State the two-part rule explicitly and require students to identify both parts out loud for a second example (fibonacci): 'What's the base case? What's the recursive case, and does it move toward the base case?'

Instructor note: *Expect real confusion here even with a strong group — this is normal and expected for a first exposure to recursion, not a sign you explained it badly. Plan to revisit the same trace-by-hand technique again in the capstone module if a recursive search is attempted there.*

5. When to use recursion vs. a loop (10 min)

- Be honest: 'Everything recursion does here, a loop could also do — factorial in Module 6 was loop-based.' Frame recursion's real value as fitting problems that are naturally defined in terms of smaller versions of themselves, which becomes more visible in later topics beyond this course (trees, divide-and-conquer algorithms).

Common Student Misconceptions

Misconception	How to Address It
"Forgetting & still lets me modify the caller's variable, just less reliably."	No — omitting & silently falls back to full pass-by-value with no error and no warning of intent; show this side by side with the reference version.
"A recursive function needs a loop inside it to actually repeat."	The repeated call to itself IS the repetition — there is no loop keyword involved at all in most simple recursive functions.
"Recursion is always better/more advanced than a loop, so I should prefer it."	Reframe: for simple counting/accumulating problems, a loop is often just as clear and slightly more efficient — recursion earns its place on problems naturally defined recursively.

Exercises & Module Project

Exercise: 07-functions-ii/exercises/exercise1.cpp — swapValues (reference), overloaded maxOf, recursive sumUpTo. Project: a Recursive Toolkit (power, sumDigits, isPalindrome) plus Rock-Paper-Scissors refactored into functions using references for score tracking (07-functions-ii/project/README.md).

Facilitation guidance: Require students to write, in a comment, the base case and recursive case for each recursive function BEFORE writing the code — this forces the two-part identification habit from the lesson rather than letting students guess-and-check their way to working recursion.

Discussion & Comprehension Check Questions

1. Why can a function only return one value, but modify several through reference parameters?

Guidance: return hands back exactly one value and ends the function; reference parameters instead give the function direct write-access to multiple existing caller variables.

2. What happens if a recursive function's base case can never be reached for some input?

Guidance: The function calls itself indefinitely until the program crashes (stack overflow) — always verify the base case is reachable for every valid input.

Assessment Ideas

- Hand-trace assessment: give a small recursive function (not factorial, to avoid rote memorization) and require a full step-by-step expansion, matching the board demonstration format.
- Bug hunt: provide a recursive function with a base case that's never reached (e.g. off-by-one in the base condition) and ask students to identify and fix it.

Answer Key Notes

See 07-functions-ii/exercises/solutions/exercise1_solution.cpp.

Wrap-Up & Bridge to Next Module

Bridge forward: "Every program so far manipulates data that's separate from the logic acting on it. Next module introduces classes — bundling data and the functions that act on it into a single, self-contained type. This is the foundation of object-oriented programming."

Instructor Tips

- If forced to cut content for time, cut default parameters or overloading depth before cutting recursion time — recursion is the concept students most need repeated, unhurried exposure to.
- Consider a short 5-minute recursion re-cap at the START of Module 8, since spaced repetition meaningfully helps this specific topic stick.